Dokumentacija i objašnjenje rješenja

Kolegij: Web programiranje • Zadatak: Probni kolokvij 1

Bootstrap 5.3 Sass jQuery Node.js Express Vite Dev Server

1. Pregled zadataka i bodovnog praga

Zadatak	Bodovi	Opis i zahtjevi
Zadatak 1	21 bod	Prilagodba Bootstrap boja (success, warning, danger) i fonta (Fraunces). Izrada navigacijske trake sa sjenom i ikonom `stack`. Dodavanje UUID-a u `todos.js`, izvoz modula. Učitavanje u `index.js` poslužitelja, Express rute (GET, PUT, DELETE). Definiranje funkcije `get_todos` na klijentu.
Zadatak 2	16 bodova	Izrada prototipa kartice u `index.html` (naslov, datum, status, gumbi). Oblikovanje kartice (zaobljenost `rounded-4`, gumbi pilule). Responzivni raspored (3 stupca na velikom, 2 na srednjem, 1 na malom ekranu). Komentiranje prototipa radi prelaska na dinamičko renderiranje.
Zadatak 3	22 boda	Funkcija `render_todos` za dinamičko iscrtavanje kartica. Funkcija `toggle` za promjenu statusa (server + klijent, zamjena ikone). Funkcija `remove` za brisanje (server + klijent, uklanjanje kartice). Formatiranje datuma iz Unix milisekundi u `M/D/YYYY`. Implementacija fade-in efekta pri inicijalnom učitavanju.
Urednost koda	1 bod	Uklanjanje svih `node_modules` mapa prije komprimiranja u zip.

2. Detaljan pregled svih izmjena po datotekama

📁 template/server/todos.js (Poslužiteljska baza podataka)

Što je promijenjeno:

- Uvezen je `randomUUID` iz ugrađenog Node.js modula `crypto`.
- Svakom zapisu u nizu `todos` generiran je jedinstveni UUID atribut: `id: randomUUID()`.
- Niz je eksportiran: `export { todos }; export default todos;`.

Zašto:

U originalnom predlošku zapisi nisu imali identifikatore niti je modul bio eksportiran. Za ažuriranje i brisanje točno određenog zapisa, poslužitelj i klijent moraju komunicirati putem jedinstvenog ključa (UUID). Ovo donosi **2 boda** za UUID i omogućuje uvoz u poslužiteljski modul.

```
import { randomUUID } from "crypto";

const todos = [
  { id: randomUUID(), title: "Buy groceries", timestamp: 1764845228329, done: true },
  { id: randomUUID(), title: "Email client",  timestamp: 1764845228329, done: true },
  ...
];

export { todos };
export default todos;
```

📁 template/server/index.js (Glavni modul poslužitelja)

Što je promijenjeno:

- Uvezen niz `todos` iz `./todos.js`.
- Dodan middleware `app.use(express.json());` za prihvaćanje JSON tijela zahtjeva.
- Definirana ruta za dohvaćanje: `GET /api/todos`.
- Definirana ruta za uređivanje: `PUT /api/todos/:id` (traži element po UUID-u i ažurira `done`).
- Definirana ruta za brisanje: `DELETE /api/todos/:id` (uklanja zapis pomoću `splice`).

Zašto:

Zadatak 1 zahtijeva REST API rute za dohvat, uređivanje i brisanje. `express.json()` je ključan jer klijentska `toggle` funkcija šalje JSON tijelo (`{ done: true/false }`) koje Express bez ovog parsera ne može pročitati.

```
import express from 'express';
import todos from './todos.js';

const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

// Dohvaćanje zapisa
app.get("/api/todos", (req, res) => res.json(todos));

// Uređivanje postojećeg zapisa (toggle statusa)
app.put("/api/todos/:id", (req, res) => {
    const todo = todos.find(t => t.id === req.params.id);
    if (!todo) return res.status(404).json({ error: "Not found" });
    if (req.body.done !== undefined) todo.done = req.body.done;
    res.json(todo);
});

// Brisanje postojećeg zapisa
app.delete("/api/todos/:id", (req, res) => {
    const index = todos.findIndex(t => t.id === req.params.id);
    if (index === -1) return res.status(404).json({ error: "Not found" });
    const [removed] = todos.splice(index, 1);
    res.json(removed);
});
```

📁 template/src/scss/style.scss (Stilovi, Bootstrap varijable i tipografija)

Što je promijenjeno:

- Prilagođene Bootstrap Sass varijable: `$success: #3cb47c;`, `$warning: #ffea63;`, `$danger: #fe604e;`.
- Varijable su deklarirane **prije** uvoza `@import "bootstrap/scss/bootstrap";`.
- Uvezen Google font **Fraunces** i postavljen na `body { font-family: "Fraunces", serif; }`.

Zašto:

U Sass sustavu varijable sa zastavicom `!default` moraju se definirati prije uvoza Bootstrapa kako bi prebrisale zadane vrijednosti. Time gumbi i ikone uspjeha/upozorenja/greške poprimaju točne tražene hex boje, a font Fraunces daje izgled iz video prikaza.

```
$primary: #26a69a;
$light: #f8f9fa;
$dark: #212529;
$success: #3cb47c;
$warning: #ffea63;
$danger: #fe604e;

@import "bootstrap/scss/bootstrap";

@import url('https://fonts.googleapis.com/css2?
family=Fraunces:ital,opsz,wght@0,9..144,100..900;1,9..144,100..900&display=swap');
@import url("https://cdn.jsdelivr.net/npm/bootstrap-icons@1.13.2/font/bootstrap-icons.min.css");

body {
  font-family: "Fraunces", serif;
  font-optical-sizing: auto;
  font-style: normal;
}
```

📁 template/src/index.html (HTML struktura i prototip kartice)

Što je promijenjeno:

- Navigacija: zelena pozadina (`bg-success`), sjena (`shadow`), ikona `bi-stack text-warning` i naslov `TaskProgress` .
- Prototip kartice: definiran u HTML-u sa zaobljenošću `rounded-4` , sjenom `shadow-sm` , obrubom `border` i gumbima u obliku pilula (`rounded-pill flex-fill`), a zatim **zakomentiran**.
- Kontejner: `<div id="todos-container" class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4 w-100"></div>` .

Zašto:

Zadatak 2 traži izradu i stiliziranje HTML prototipa, a zatim njegovo komentiranje prije dinamičkog renderiranja. Klase `row-cols-1 row-cols-md-2 row-cols-lg-3` pružaju responzivni prikaz: 3 kartice na desktopu, 2 na tabletu, 1 na mobitelu uz razmak `g-4` .

```
<nav class="navbar bg-success shadow mb-4">
    <div class="container">
        <a class="navbar-brand text-white d-flex align-items-center" href="#">
            <i class="bi bi-stack text-warning me-2 fs-5"></i>
            <span>TaskProgress</span>
        </a>
    </div>
</nav>

<main class="container mb-5">
    <!-- Prototip kartice (Zadatak 2):
    <div class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4">
        <div class="col">
            <div class="card rounded-4 border shadow-sm p-3 h-100">
                <div class="card-body d-flex flex-column justify-content-between p-0">
                    <div>
                        <h5 class="card-title fw-bold mb-2">Buy groceries</h5>
                        <p class="card-text mb-1"><span class="fw-bold">When:</span> 12/4/2025</p>
                        <p class="card-text mb-3"><span class="fw-bold">Status:</span> <i class="bi bi-check-
circle-fill text-success"></i></p>
                    </div>
                    <div class="d-flex gap-2 mt-auto">
                        <button class="btn btn-outline-secondary rounded-pill flex-fill">Toggle</button>
                        <button class="btn btn-outline-secondary rounded-pill flex-fill">Remove</button>
                    </div>
                </div>
            </div>
        </div>
    </div>
    -->

    <div id="todos-container" class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4 w-100"></div>
</main>
```

📁 template/src/js/index.js (Klijentska logika, animacije i API pozivi)

Što je implementirano:

- **`format_date(timestamp)`:** pretvara Unix milisekunde u format `M/D/YYYY` (npr. `12/4/2025`).
- **`get_todos(callback)`:** asinkroni dohvat podataka preko jQueryja (`$.get('/api/todos')`) koji vraća Promise.
- **`toggle(id)`:** šalje `PUT /api/todos/${id}` sa suprotnim `done` statusom i mijenja ikonu (`check-circle-fill` ↔ `x-circle-fill`).
- **`remove(id)`:** šalje `DELETE /api/todos/${id}` i uklanja karticu uz `.fadeOut(300)` animaciju.
- **`render_todos(todos)`:** stvara DOM elemente kartica, povezuje event listenere i pokreće `.fadeIn(500)` pri učitavanju.
- **Globalno izlaganje:** funkcije su postavljene na `window` objekt radi automatskog testiranja.

```
import $ from "jquery";

function format_date(timestamp) {
    const d = new Date(timestamp);
    return `${d.getMonth() + 1}/${d.getDate()}/${d.getFullYear()}`;
}

function get_todos(callback) {
    const request = $.get("/api/todos");
    if (typeof callback === "function") request.then(callback);
    return request;
}

function toggle(id) {
    const $col = $(`[data-id="${id}"]`);
    const $statusIcon = $col.find(".status-icon");
    const isDone = $statusIcon.hasClass("bi-check-circle-fill");
    const newStatus = !isDone;

    return $.ajax({
        url: `/api/todos/${id}`,
        method: "PUT",
        contentType: "application/json",
        data: JSON.stringify({ done: newStatus }),
        success: (updated) => {
            if (updated.done) {
                $statusIcon.removeClass("bi-x-circle-fill text-danger").addClass("bi-check-circle-fill text-
success");
            } else {
                $statusIcon.removeClass("bi-check-circle-fill text-success").addClass("bi-x-circle-fill text-
danger");
            }
        }
    });
}

function remove(id) {
    const $col = $(`[data-id="${id}"]`);
    return $.ajax({
        url: `/api/todos/${id}`,
        method: "DELETE",
        success: () => {
            $col.fadeOut(300, function () { $(this).remove(); });
        }
    });
}

function render_todos(todos) {
    const $container = $("#todos-container");
    $container.empty();

    todos.forEach((todo) => {
        const formattedDate = format_date(todo.timestamp);
        const iconClass = todo.done ? "bi-check-circle-fill text-success" : "bi-x-circle-fill text-danger";

        const $col = $(`
            <div class="col" data-id="${todo.id}">
                <div class="card rounded-4 border shadow-sm p-3 h-100">
                    <div class="card-body d-flex flex-column justify-content-between p-0">
                        <div>
                            <h5 class="card-title fw-bold mb-2">${todo.title}</h5>
                            <p class="card-text mb-1"><span class="fw-bold">When:</span> ${formattedDate}</p>
                            <p class="card-text mb-3"><span class="fw-bold">Status:</span> <i class="bi
${iconClass} status-icon"></i></p>
                        </div>
                        <div class="d-flex gap-2 mt-auto">
                            <button class="btn btn-outline-secondary rounded-pill flex-fill btn-
toggle">Toggle</button>
                            <button class="btn btn-outline-secondary rounded-pill flex-fill btn-
remove">Remove</button>
```

```
                        </div>
                    </div>
                </div>
            </div>
        `);

        $col.find(".btn-toggle").on("click", () => toggle(todo.id));
        $col.find(".btn-remove").on("click", () => remove(todo.id));

        $col.hide();
        $container.append($col);
        $col.fadeIn(500);
    });
}

window.format_date = format_date;
window.get_todos = get_todos;
window.toggle = toggle;
window.remove = remove;
window.render_todos = render_todos;

$(() => {
    get_todos().then((todos) => render_todos(todos));
});
```

3. Upute za pokretanje i predaju rješenja

> ⚠️ **Važno pravilo prije predaje (1 bod):**
> Uklonite sve mape `node_modules` (i u korijenu i u `server/` mapi) prije komprimiranja u `prezime_ime_kolokvij_1.zip` arhivu kako bi datoteka bila lagana za slanje i zadovoljila uvjet kolokvija.

Pokretanje poslužitelja (port 3000):

```
cd C:\Users\borna\Downloads\template_kol\template\server
npm install
node index.js   # ili npm run nodemon
```

Pokretanje klijenta (port 8080):

```
cd C:\Users\borna\Downloads\template_kol\template
npm install
npx vite
```

Aplikacija se u pregledniku otvara na adresi: **`http://localhost:8080/index.html`**.